

Agentic AI Loan Processing

Interview-Ready Project Handbook

Project focus

Personal-loan POC using a central LLM orchestrator, LangGraph stateful workflow, LangChain tools, specialised domain agents, human escalation, and resumable session persistence.

Prepared from the supplied Agentic AI Release Sep-23 notes.

Below is the interview-ready explanation for Agentic AI Release Sep-23.

1. One-minute project explanation

"This is an agentic AI loan-processing proof of concept for personal loans. It collects customer information, validates PAN/Aadhaar, retrieves customer and credit data, extracts salary details from a PDF, checks loan-purpose and city policies, performs a risk assessment, presents a loan agreement, and saves the full session. A central orchestrator uses an LLM to decide which specialised tool to call next. If the system cannot understand or validate customer information after retries, it escalates the case to a human operator. The workflow is stateful, so an interrupted loan application can be resumed."

2. The complete business flow

```
Customer loan request
↓
Collect loan purpose
↓
Check purpose policy
↓
Collect loan amount and PAN/Aadhaar
↓
Validate identity and fetch existing customer data
↓
Get salary information
├ Existing customer: use/update existing salary
└ New customer: upload salary PDF and extract information
↓
Collect city
↓
Check city + purpose + amount policy
↓
Assess credit and affordability risk
↓
If approved: present agreement and capture acceptance
↓
Save session / complete application
```

Example:

"I need Rs. 5 lakh for home renovation."

1. System identifies the loan purpose: home renovation.
2. Purpose-policy tool confirms whether this category is allowed.
3. System asks loan amount and PAN/Aadhaar.
4. It finds the customer record and retrieves salary, EMI, and credit details.
5. It asks whether the salary data should be updated.
6. If yes, customer provides a salary PDF.
7. Salary agent extracts income from the PDF.
8. System asks for the customer's city.
9. Geo-policy agent checks whether Rs. 5 lakh for that purpose is permitted in that city.
10. Risk agent checks income, existing EMI, credit score, and requested amount.
11. If eligible, agreement agent calculates EMI, tenure, fees, and asks the customer to accept.

3. Architecture used

```
CLI / Streamlit UI / Human Dashboard
↓
```



The project has these major layers:

- Presentation layer
- CLI: `run_loan_cli.py`
- Streamlit UI
- Human-operator dashboard
- FastAPI multi-turn API
- Orchestration layer
- `LoanAgentOrchestrator`
- Controls the end-to-end loan workflow.
- Tracks what is complete and what data is still missing.
- AI and agent layer
- LLM decides which tool to call next.
- Domain agents perform specialised work such as risk assessment or salary extraction.
- Integration/data layer
- Aadhaar verification API
- Credit-score API
- Local sample loan CSV
- Salary PDF extraction
- Persistence layer
- `session_data/*.json` stores active or completed loan applications.
- `escalation_data/*.json` stores human-escalation tickets and responses.
- Deployment layer
- Docker, Docker Compose, Kubernetes YAML files, and Kong API Gateway configurations are included for microservice deployment.

4. Why LangChain is used

LangChain provides the building blocks for interacting with an LLM.

In this project, it is used for:

- Prompt templates
- LLM wrapper/provider access
- Tool definitions
- Structured tool actions
- Agent action/result objects

For example, the LLM is given a prompt containing:

- Customer request
- Data already collected
- Previous tool results
- Available tools
- Instructions about the next required step

The LLM replies in a format similar to:

```
Thought: I need to verify the customer identity first.
Action: DataQuery
Action Input: ABCDE1234F
```

The project parses that response and runs the selected tool.

Important interview answer:

“LangChain is used as the LLM integration and tool abstraction layer. It allows the LLM to select a registered business tool instead of only generating free-text replies.”

5. Why LangGraph is used

LangGraph controls the workflow state and execution loop.

LangChain alone can call tools, but a banking process needs stronger workflow control:

- What has already been completed?
- Which fields are still missing?
- Did the tool fail?
- Has the customer accepted the agreement?
- Has the workflow looped too many times?
- Is the application finished?

LangGraph stores this in `AgentState`.

Important state fields include:

```
purpose
amount
city
pan
aadhaar
document_path
salary_update_confirmation
agreement_response
steps_completed
intermediate_steps
step_count
workflow_finished
```

The basic LangGraph execution loop is:

```
run_agent
↓
LLM chooses an action
↓
execute_tools
↓
Tool result updates AgentState
↓
```

```
run_agent again
↓
Workflow finishes only when required steps are complete
```

Why this matters:

“LangGraph gives the project memory and control. It prevents the AI from repeatedly asking the same question, tracks mandatory loan-processing steps, limits loops, and lets the workflow resume after an interruption.”

A very important technical point:

This is not a group of agents freely chatting with one another. It is mainly a central LLM planner/orchestrator that chooses from specialised domain tools. LangGraph manages the stateful loop around that planner.

6. What the orchestrator does

The main orchestrator is:

```
LoanAgentOrchestrator
```

It creates and coordinates:

```
DataQueryAgent
GeoPolicyAgent
RiskAssessmentAgent
UserInteractionAgent or CustomerAgent
SalarySheetGeneratorAgent
SalarySheetRetrieverAgent
PDFSalaryExtractorAgent
LoanPurposeAssessmentAgent
AgreementAgent
Human escalation support
```

Its responsibilities are:

1. Start or resume a session.
2. Parse the initial customer request.
3. Maintain customer details during the journey.
4. Create the tool list.
5. Give the LLM the current state and permitted next actions.
6. Run the selected tool.
7. Update state after each tool call.
8. Save progress to disk.
9. Escalate unclear cases to a human.
10. Finish only after the required workflow steps are complete.

7. Agents and what each one does

User Interaction Agent

Purpose: asks the customer questions.

Typical questions:

- What is the loan purpose?
- What loan amount is required?
- What is your PAN or Aadhaar number?
- What city do you live in?
- Do you want to update salary details?
- Do you accept the agreement?

Why separate?

It keeps conversation handling separate from risk, KYC, and policy logic.

Loan Purpose Assessment Agent

Purpose: converts the customer's free-text reason into a standard loan category and checks its eligibility.

Example:

Customer says: "I need money for a bike."
Standard category: vehicle_purchase
Policy result: eligible / prohibited / conditional

Why separate?

Customers speak naturally, but bank policy needs standardised categories.

Data Query Agent

Purpose: finds an existing customer using PAN or Aadhaar.

It retrieves available customer information such as:

- Monthly salary
- Existing EMI
- City
- Transaction/balance details
- Identifier details

The project uses sample data for this POC.

Important point:

The data-query service intentionally excludes the credit score from its basic result. Credit information can be retrieved separately through the credit-score flow.

PAN-Aadhaar Validation Tool

Purpose: verifies that the PAN and Aadhaar information belongs to the same customer.

Why important?

It reduces identity mismatch risk and supports the KYC process.

Credit Score Tool

Purpose: obtains a credit score using PAN.

Why separate from data retrieval?

Identity/customer information and bureau credit information are different domains. In real banking, they normally come from different systems and have different controls.

PDF Salary Extractor Agent

Purpose: extracts salary information from the customer's salary PDF.

It is used:

- For a new customer with no saved salary details.
- When an existing customer chooses to update salary information.

Typical extracted fields:

Monthly salary
Existing EMI
Employer/income details
Potential credit-related details

Why important?

It automates document reading so the risk assessment gets structured income data rather than raw PDF text.

Salary Sheet Generator Agent

Purpose: fallback only.

If PDF extraction fails, it can generate mock/sample salary data for the demo flow.

Important interview honesty:

"This is a POC fallback. In a production bank, we would not create salary information. We would request a valid document or send the case for human review."

Geo Policy Agent

Purpose: checks whether the loan is allowed for the customer's location, loan purpose, and requested amount.

Input format:

city:Mumbai,purpose:personal,amount:100000

It checks:

- Is the city supported?
- Is the purpose allowed?
- Is the requested amount within policy limits?

- Are special conditions required?

Why separate?

Credit eligibility and operational/geographic policy are different decision dimensions.

Risk Assessment Agent

Purpose: evaluates affordability and credit risk.

Main inputs:

Monthly salary
Existing EMI
Credit score
Requested loan amount

It uses:

- Rule/helper logic for parsing and risk tiering
- An LLM prompt for a concise risk explanation
- Debt/EMI analysis

Typical output:

Risk category
Credit score
Income summary
Requested amount
LLM risk rationale
Approve / Conditional / Reject-style assessment

Important interview answer:

“The LLM is used to explain the risk assessment, while financial values and risk rules remain structured inputs. In production, the final lending decision should be controlled by approved lending policy and risk models, not only an LLM.”

Agreement Agent

Purpose: prepares loan terms after the risk stage and captures digital acceptance.

It calculates or presents:

- Principal amount
- Interest rate
- Loan tenure
- Monthly EMI
- Processing fee
- Total payable amount
- Repayment schedule
- Terms and conditions

Then it checks whether the customer says:

I Agree / I Accept

or

I Decline / I Reject

Why separate?

Agreement creation is a post-decision business process, not a risk-assessment task.

Human Agent / Escalation Manager

Purpose: handles cases the automated flow cannot process correctly.

Current trigger in this POC:

```
Invalid customer response
↓
Retry
↓
Retry limit reached
↓
Ask customer whether they want human help
↓
Create escalation record
↓
Human operator responds
↓
Workflow continues or ends
```

The system stores:

```
Failed agent name
Question asked
Customer input
Conversation history
Failure count
Session ID
Escalation timestamp
Priority
```

The human response is saved and returned to the workflow.

8. Tools used by the LLM planner

The LLM does not directly query databases or parse PDFs. It selects controlled tools.

```
UserInteraction
LoanPurposeAssessment
DataQuery
CreditScoreByPAN
ValidatePANAadhaar
MergeUserDataWithCredit
UseExistingUserData
GeoPolicyCheck
RiskAssessment
PDFSalaryExtractor
SalarySheetGenerator
SalarySheetRetriever
AgreementPresentation
```

The tool sequence is controlled by prompts and workflow state.

A simplified correct path is:

```
UserInteraction
→ LoanPurposeAssessment
→ UserInteraction for amount
→ UserInteraction for PAN/Aadhaar
→ DataQuery
→ Salary data flow
→ UserInteraction for city
→ GeoPolicyCheck
→ RiskAssessment
→ AgreementPresentation
→ User acceptance
```

9. Session management

Loan applications can take time. The customer may close the page or stop midway.

The project stores each loan session in a JSON file:

```
session_data/loan_<timestamp>_<id>.json
```

It saves:

- Initial loan request
- Workflow step
- Collected information
- Conversation history
- Agent state
- Orchestrator state
- Status: active, completed, interrupted, or ended by user

Why it matters:

"Session persistence prevents the customer from starting again after an interruption and helps preserve an audit trail."

10. Interview architecture explanation

Use this answer:

"The project follows a layered microservice-friendly architecture. The UI and CLI collect customer input. A FastAPI multi-turn API exposes the loan workflow. A central orchestrator uses LangGraph to control state and LangChain to let an LLM choose approved tools. Specialised agents handle KYC, customer data retrieval, document extraction, policy validation, risk assessment, agreement generation, and human escalation. External capabilities such as Aadhaar and credit checks are exposed as separate services. JSON-based session and escalation stores provide persistence for the POC. Docker, Kong, and Kubernetes files support deployment-oriented architecture."

11. Likely interview questions and strong answers

"Why did you use an agentic workflow instead of one prompt?"

"Loan processing is multi-step and regulated. A single prompt would be difficult to control, test, and audit. The agentic workflow breaks the journey into controlled tools and tracks the state of each step."

"Why use LangGraph?"

"LangGraph manages stateful execution. It tracks completed steps, collected PAN/Aadhaar, salary-document state, tool outputs, retry count, and termination conditions. That prevents repeated questions and supports resume/recovery."

"Why use LangChain?"

"LangChain provides the LLM interface, prompt templates, and tool abstractions. It lets the LLM select controlled actions such as DataQuery or RiskAssessment instead of directly performing unrestricted operations."

"What makes this system agentic?"

"It observes the current case state, chooses the next approved action, invokes a specialised tool, uses the tool output to update state, and repeats until the workflow reaches a controlled end state."

"How is KYC handled?"

"The system collects PAN/Aadhaar, validates identifier formats, queries the customer record, can validate PAN-Aadhaar consistency, and supports Aadhaar-related service integration. It also captures salary evidence through a PDF workflow."

"How does the system assess loan risk?"

"It uses structured inputs such as monthly salary, existing EMI, credit score, and loan amount. Helper logic derives risk categories and affordability signals, while the LLM creates an understandable risk rationale."

"Does the LLM approve the loan?"

"Not safely on its own. In this POC, the LLM helps choose workflow actions and generate explanations. A production lending decision must be controlled by approved policy rules, risk models, and human-review controls."

"What happens if the LLM gives invalid output?"

"The code parses expected Action and Action Input fields, rejects unparseable or hallucinated observations, limits the number of steps, and terminates safely when it cannot continue."

"How do you prevent infinite loops?"

"The workflow tracks step_count, maintains steps_completed, and has termination thresholds. It also sets workflow_finished after completion."

“How does human-in-the-loop work?”

“If an agent repeatedly cannot process a user response, the system offers human escalation. It saves the full context as an escalation record, accepts an operator response via dashboard/API, and continues the session using that response.”

“What data is persisted?”

“The POC persists session state, collected data, conversation history, workflow progress, and escalation context in JSON files. In production this should move to encrypted, access-controlled persistent storage.”

“What would you change for production?”

“I would add secure databases, authentication and RBAC, encryption, real KYC/credit integrations, immutable audit logs, policy versioning, observability, model monitoring, consent controls, stronger exception handling, and formal lending-model governance.”

“What is the role of microservices?”

“The project separates Aadhaar verification, credit-score lookup, human-agent API, and multi-turn workflow API. This allows each capability to be independently deployed, scaled, secured, and replaced.”

12. Important limitations to say honestly

Do not overclaim.

Say:

“This is a POC. It demonstrates workflow orchestration, tool use, KYC-style checks, risk explanation, session persistence, and escalation. It uses sample data and simulated services in parts of the flow. It is not a production loan-origination system and should not autonomously approve real loans without bank-approved policies, integrations, controls, and human governance.”

That answer makes you sound practical and trustworthy in an interview.